

Digital Signals

- **Digital Signals have two basic states:**
 - 1 (logic “high”, or H, or “on”)
 - 0 (logic “low”, or L, or “off”)
- **Digital values are in a *binary* format.**
Binary means 2 states.
- **A good example of binary is a light**
(only on or off)



In Binary, there are only 0's and 1's. These numbers are called "Base-2" (Example: 010_2)

Binary to Decimal

Base 2 = Base 10

000 = 0

001 = 1

010 = 2

011 = 3

100 = 4

101 = 5

110 = 6

111 = 7

**We count in "Base-10"
(0 to 9)**

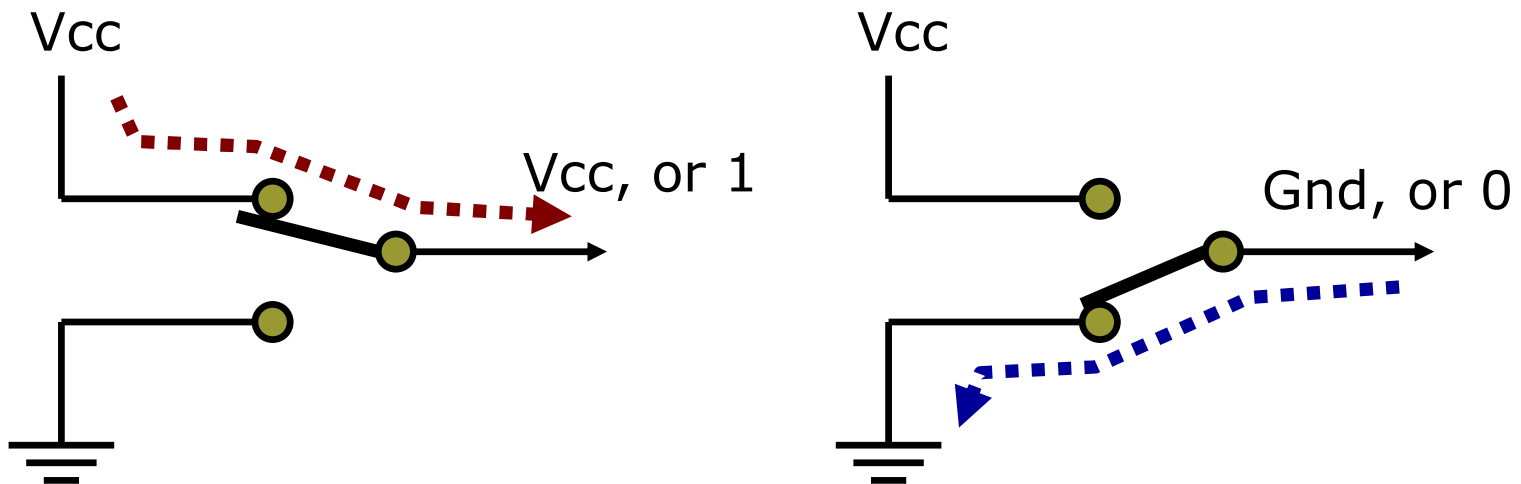
Binary as a Voltage

- **Voltages are used to represent logic values:**
- **A voltage present (called Vcc or Vdd) = 1**
- **Zero Volts or ground (called gnd or Vss) = 0**

A simple switch can provide a logic high or a logic low.

A Simple Switch

- Here is a simple switch used to provide a logic value:



There are other ways to connect a switch.

Number System

The number system is the system of naming or representing numbers.

**There are various types of number systems –
Binary, Decimal, Octal & Hexadecimal**

The important parameter of Number systems is Radix or Base which is denoted by r .

The radix or base is the number of unique digits within the number system, including the digit zero, used to represent numbers.

For Binary number system, $r=2$, (0 , 1)

For Decimal number system, $r=10$ (0, 1, 2, 3, 4, 5, 6, 7, 8, 9)

Number System

The number system is the system of naming or representing numbers.

**There are various types of number systems –
Binary, Decimal, Octal & Hexadecimal**

The important parameter of Number systems is Radix or Base which is denoted by r .

The radix or base is the number of unique digits within the number system, including the digit zero, used to represent numbers.

**For Octal number system, $r=8$
(0, 1, 2, 3, 4, 5, 6, 7)**

**For Hexadecimal number system, $r=16$
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F)**

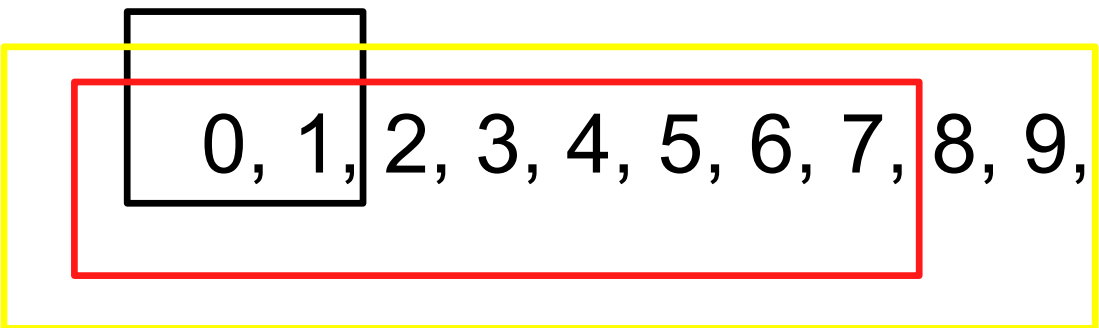
Number System

**For Binary number system, $r=2$,
(0 , 1)**

**For Octal number system, $r=8$
(0, 1, 2, 3, 4, 5, 6, 7)**

**For Decimal number system, $r=10$
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9)**

**For Hexadecimal number system, $r=16$
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F)**



0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F

Number System

Decimal	Binary	Octal	Hex
00	0000	00	0
01	0001	01	1
02	0010	02	2
03	0011	03	3
04	0100	04	4
05	0101	05	5
06	0110	06	6
07	0111	07	7
08	1000	10	8
09	1001	11	9
10	1010	12	A
11	1011	13	B
12	1100	14	C
13	1101	15	D
14	1110	16	E
15	1111	17	F

Review of Number systems

Memorize the first ten powers of two

Table 1.2 Powers of 2.

n	2^n	n	2^n
1	2	11	2,048
2	4	12	4,096
3	8	13	8,192
4	16	14	16,384
5	32	15	32,768
6	64	16	65,536
7	128	17	131,072
8	256	18	262,144
9	512	19	524,288
10	1,024	20	1,048,576

Number System

Any number to decimal conversion

$$N = a_{n-1}r^{n-1} + \dots + a_1r^1 + a_0r^0$$

where, for $n = 0, 1, 2, 3, \dots$

$$(101111)_2 = (?)_{10}$$

Number systems

- Converting to decimal from binary:

$$N = a_{n-1}r^{n-1} + \dots + a_1r^1 + a_0r^0$$

where, for $n = 0, 1, 2, 3, \dots$

5	4	3	2	1	0
1	0	1	1	1	1 ₂



$$\begin{aligned}
 & \boxed{1 \cdot 2^5} + \boxed{0 \cdot 2^4} + \boxed{1 \cdot 2^3} + \boxed{1 \cdot 2^2} + \\
 & \boxed{1 \cdot 2^1} + \boxed{1 \cdot 2^0} = \boxed{47_{10}}
 \end{aligned}$$

- **Convert to decimal from binary:**

– 1011011

a. 27

b. 91

c. 109

d. -109

e. 551

$$1 \times 2^6 + 0 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

=

$$1 \times 64 + 0 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1$$

= 91

$$N = a_{n-1}r^{n-1} + \dots + a_1r^1 + a_0r^0$$

where, for $n = 0, 1, 2, 3, \dots$

Number systems

- Convert to decimal from Hexadecimal

:

– 2EB

a. 27

b. 91

c. 109

d. 551

e. 747

$$2 \times 16^2 + 14 \times 16^1 + 11 \times 16^0$$

=

$$2 \times 256 + 14 \times 16 + 11 \times 1 = 512 + 224 + 11$$

$$= 747$$

$$N = a_{n-1}r^{n-1} + \dots + a_1r^1 + a_0r^0$$

where, for $n = 0, 1, 2, 3, \dots$

Number systems

$$(41.6875)_{10} = (?)_2$$

Number systems

$$(41.6875)_{10} = (?)_2$$

41

0.6875

Number systems

$$(41.6875)_{10} = (?)_2$$

A handwritten diagram on a light yellow background showing the conversion of 41.6875 to binary. It consists of two parts: an integer division and a fractional multiplication. The integer part shows 41 divided by 2, with remainders 1, 0, 0, 1, 0, 1 written in blue circles to the right of the quotient. The fractional part shows 0.6875 multiplied by 2, with the integer parts of the results (0, 1, 0, 1) written in blue circles to the right. The final binary result 101001.1001 is not explicitly written.

$$\begin{array}{r|l} 2 & 41 \\ \hline 2 & 20 \\ \hline 2 & 10 \\ \hline 2 & 5 \\ \hline 2 & 2 \\ \hline & 1 \end{array} \begin{array}{l} \textcircled{1} \\ \textcircled{0} \\ \textcircled{0} \\ \textcircled{1} \\ \textcircled{0} \\ \textcircled{1} \end{array}$$

$$0.6875 \times 2 =$$

101001

Number systems

$$(41.6875)_{10} = (?)_2$$

$$\begin{array}{r|l} 2 & 41 \\ \hline 2 & 20 \\ \hline 2 & 10 \\ \hline 2 & 5 \\ \hline 2 & 2 \\ \hline & 1 \end{array}$$

$$0.6875 \times 2 = 1.3750$$

101001.1

Number systems

$$(41.6875)_{10} = (?)_2$$

2		41	①
2		20	①
2		10	①
2		5	①
2		2	①
			①

$$0.6875 \times 2 = 1.3750$$

$$0.3750 \times 2 = 0.750$$

101001.10



Number systems

$$(41.6875)_{10} = (?)_2$$

A handwritten binary conversion of the decimal number 41.6875. The integer part 41 is divided by 2 repeatedly, with remainders 1, 0, 0, 1, 0, 1 written to the right of the division lines. The fractional part 0.6875 is multiplied by 2 repeatedly, with integer parts 1, 0, 1, 0 written above the multiplication lines. The final binary result is 101001.101.

$$\begin{array}{r} 2 \overline{) 41} \text{ (1)} \\ 2 \overline{) 20} \text{ (0)} \\ 2 \overline{) 10} \text{ (0)} \\ 2 \overline{) 5} \text{ (1)} \\ 2 \overline{) 2} \text{ (0)} \\ \text{(1)} \end{array}$$

$$0.6875 \times 2 = 1.3750$$

$$0.3750 \times 2 = 0.750$$

$$0.750 \times 2 = 1.50$$

101001.101

Number systems

$$(41.6875)_{10} = (?)_2$$

$$\begin{array}{r|l} 2 & 41 \\ \hline 2 & 20 \\ \hline 2 & 10 \\ \hline 2 & 5 \\ \hline 2 & 2 \\ \hline & 1 \end{array}$$

$$0.6875 \times 2 = 1.3750$$

$$0.3750 \times 2 = 0.750$$

$$0.750 \times 2 = 1.50$$

$$0.5 \times 2 = 1.0$$

101001.1011

Number systems

$$(41.6875)_{10} = (?)_2$$

$$\begin{array}{r|l} 2 & 41 \\ \hline 2 & 20 \\ \hline 2 & 10 \\ \hline 2 & 5 \\ \hline 2 & 2 \\ \hline & 1 \end{array}$$

$$0.6875 \times 2 = 1.3750$$

$$0.3750 \times 2 = 0.750$$

$$0.750 \times 2 = 1.50$$

$$0.5 \times 2 = 1.0$$

$$101001.1011 = (41.6875)_{10}$$

Number systems

$$(101001.1011)_2 = (??)_{10}$$

Number systems

$$(101001.1011)_2 = (41.6875)_{10}$$

Number systems

$$(101001.1011)_2 = (??)_{10}$$

$$N = a_{n-1}r^{n-1} + a_{n-2}r^{n-2} + \dots$$

$$\dots + a_1r^1 + a_0r^0 + a_{-1}r^{-1} + a_{-2}r^{-2} + a_{-3}r^{-3} + \dots$$

$$N = a_{n-1}r^{n-1} + \dots + a_1r^1 + a_0r^0$$

where, for $n = 0, 1, 2, 3, \dots$

Number systems

$$(101001.1011)_2 = (??)_{10}$$

$$N = a_{n-1}r^{n-1} + a_{n-2}r^{n-2} + \dots$$

$$\dots + a_1r^1 + a_0r^0 + a_{-1}r^{-1} + a_{-2}r^{-2} + a_{-3}r^{-3} + \dots$$

$$N = 1 \times 2^{6-1} + 0 \times 2^{6-2} + 1 \times 2^{6-3} + 0 \times 2^{6-4} + 0 \times 2^{6-5} \\ + 1 \times 2^{6-6} + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$$

Number systems

$$(101001.1011)_2 = (??)_{10}$$

$$N = a_{n-1}r^{n-1} + a_{n-2}r^{n-2} + \dots \\ \dots + a_1r^1 + a_0r^0 + a_{-1}r^{-1} + a_{-2}r^{-2} + a_{-3}r^{-3} + \dots$$

$$N = 1 \times 2^{6-1} + 0 \times 2^{6-2} + 1 \times 2^{6-3} + 0 \times 2^{6-4} + 0 \times 2^{6-5} \\ + 1 \times 2^{6-6} + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$$

$$N = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$$

Number systems

$$(101001.1011)_2 = (??)_{10}$$

$$N = 1 \times 2^5 + 0 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 \\ + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$$

$$N = 32 + 0 + 8 + 0 + 0 + 1 + 0.5 + 0 \times 0.25 + \\ 1 \times 0.125 + 1 \times 0.0625$$

$$N = 41 + 0.6875 = 41.6875$$

$$(101001.1011)_2 = (41.6875)_{10}$$

Number System

$$\begin{array}{r} 8 \overline{) 41} \textcircled{1} \cdot 0.6875 \\ \underline{\textcircled{5}} \\ \textcircled{5} 5000 \\ \underline{ \times 8} \\ \textcircled{4} 0 \end{array}$$

$$\begin{array}{r} 16 \overline{) 41} \textcircled{9} \\ \underline{ \textcircled{2}} \\ 4.1250 \\ + 6.8750 \\ \hline \textcircled{11}.0000 \end{array}$$

Octals – Base 8

- Same steps as for conversion as binary and hexadecimal and any other base
- **Converting to octal from decimal:**
 - Divide the decimal number by 8 repeatedly.
 - The remainder gives the digits of the binary number
- Example: Convert 153_{10} to base 8.

Number systems

- **Convert to octal from decimal:**

▪ 15

a. 71

b. 177

c. F7

d. 17

e. 27

Addition and Subtraction

Binary addition is performed in the same manner as decimal addition. The complete table for binary addition is as follows:

$$0+0 = 0$$

$$0+1 = 1$$

$$1+0 = 1$$

$$1+1 = 0 \quad \text{Plus a carry over of 1}$$

Decimal
5
6
11

Binary
101
110
1011

Decimal
$3 \frac{1}{4}$
$5 \frac{3}{4}$
9

Binary
11.01
101.11
1001.00

Addition and Subtraction

Subtraction is the inverse operation of addition. To subtract, it is necessary to establish a procedure for subtracting a larger from a smaller digit. The only case in which this occurs with binary numbers is when 1 is subtracted from 0. It is necessary to borrow 1 from the next column to the left.

$$0-0 = 0$$

$$0-1 = 1 \quad \text{With borrow of 1}$$

$$1-0 = 1$$

$$1-1 = 0$$

Decimal

$$\begin{array}{r} 9 \\ -5 \\ \hline 4 \\ \hline \end{array}$$

Binary

$$\begin{array}{r} 1001 \\ -101 \\ \hline 100 \\ \hline \end{array}$$

Decimal

$$\begin{array}{r} 16 \\ -3 \\ \hline 13 \\ \hline \end{array}$$

Binary

$$\begin{array}{r} 10000 \\ -11 \\ \hline 1111 \\ \hline \end{array}$$

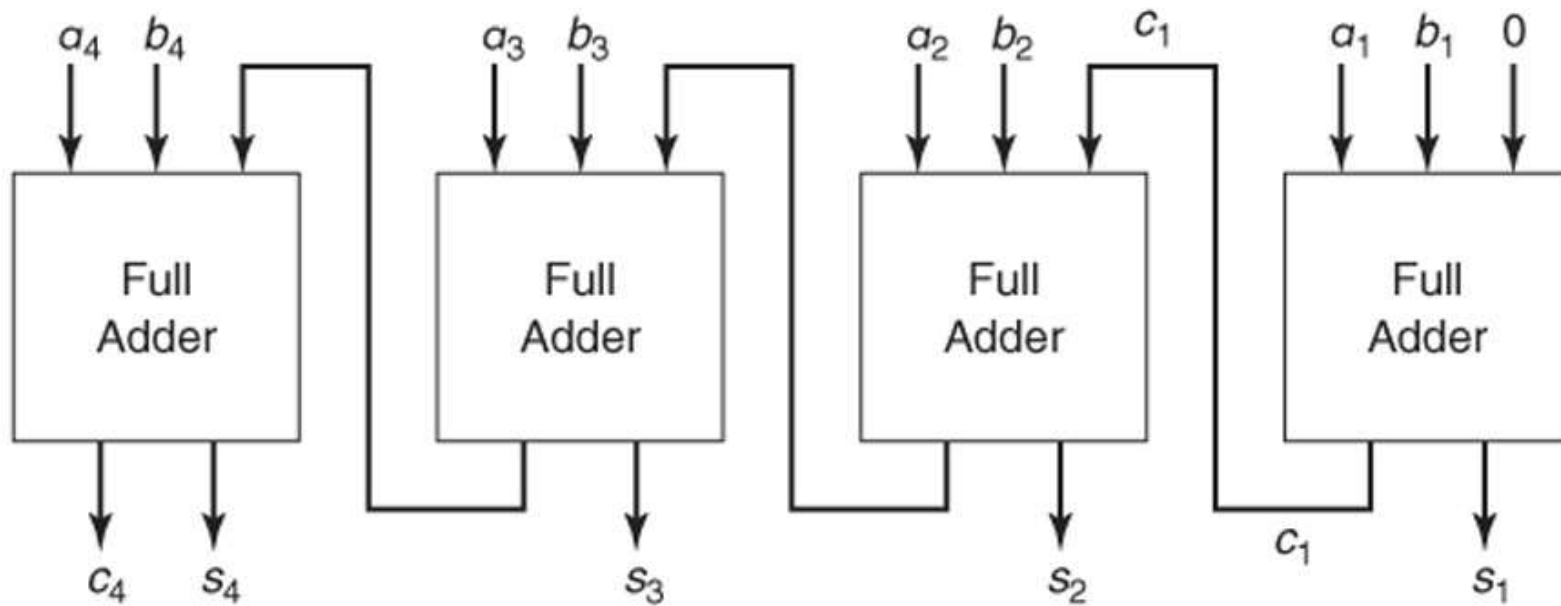
Binary Addition

Table 1.5 One-bit adder.

a	b	c_{in}	c_{out}	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

Binary Addition

Figure 1.2 A 4-bit adder.



Signed Numbers

- **Signed numbers are mostly stored in *two's complements* form**
 - Leading bit is 0 for positive numbers and 1 for negative
 - For n bits, the range of numbers that can be stored is:
 - $-2^{n-1}: 2^{n-1}-1$
- **To derive the binary negative (two's complement) of a number:**
 - Determine the magnitude (how many bits)
 - Find the binary equivalent of the magnitude
 - Complement each bit
 - Add 1

Signed Numbers

- **Example:**
 - **Derive the 6-bit binary negative (two's complement) of 17**
 - Determine the magnitude (how many bits)
 - 6bits
 - Find the binary equivalent of the magnitude
 - 010001
 - Complement each bit
 - 101110
 - Add 1
 - 101111

Table 1.6 Signed and unsigned 4-bit numbers.

Binary	Positive	Signed (two's complement)
0000	0	0
0001	1	+1
0010	2	+2
0011	3	+3
0100	4	+4
0101	5	+5
0110	6	+6
0111	7	+7
1000	8	-8
1001	9	-7
1010	10	-6
1011	11	-5
1100	12	-4
1101	13	-3
1110	14	-2
1111	15	-1

Signed Numbers

- Derive the 5-bit binary negative (two's complement) of 17
 - a. 0101111
 - b. 101111
 - c. 10000
 - d. 01111
 - e. 01110

Overflow

- **This occurs when the sum is out of range**
- **Example: for 4-bit numbers, the range is [- 8:7]**
 - Find the sum of +4 and +5
 - Find the sum of -4 and -5
- **Addition of two numbers of the opposite sign never produces overflow**
- **Adding two same-signed numbers and obtaining a result of the opposite sign indicates overflow**

Overflow

- For each of the following problems, enter **A** if the result is an overflow and **B** if it's not. Assume the number of bits is 6

1. $15 + 17$

2. $-15 + 17$

3. $-15 - 17$

4. $2 - 3$

Binary Subtraction

- **Take the two complement of the second operand**
- **Then add**
- **For signed numbers:**
 - Ignore the carry-out of the higher order
 - If two numbers of the same sign are added, and a result of the opposite sign is obtained, there's an overflow
 - Ex: $7 - 5$; $-7 - 5$
- **For Unsigned number**
 - A carry-out of zero in the higher-order bit indicates overflow
 - Ex: $5 - 7$

Binary Subtraction

- What is the 5-bit binary representation of $8 - 15$
 - a. 10111
 - b. 11000
 - c. 01001
 - d. 11001
 - e. overflow

Fractions

- Converting fractions to decimal from binary:

$$a_{-1}r^{-1} + a_{-2}r^{-2} + a_{-3}r^{-3} \dots$$

- Example

. 1 0 1₂



$$1 \cdot 2^{-1} + 0 \cdot 2^{-2} + 1 \cdot 2^{-3} = .625_{10}$$

Fractions

- Convert $.0111_2$ to decimal
 - a. .875
 - b. .375
 - c. .4375
 - d. .0700
 - e. 4.375

Fractions

- **Converting to binary from decimal:**

- Multiply the decimal number by 2 repeatedly.
- Use the integer part as the next digit each time, and then discard the integer
- When the fraction part is zero, we have an exact conversion
- Add trailing zeros to obtain the desired size

.625*2 = 1.25	.1
.25*2 = 0.50	.10
.5*2 = 1.00	.101

- For some fractions, we never get an exact conversion because the fraction parts repeats, example: .3

Examples

- **Convert the following to base 2 : $.75_{10}$**
 - a. $.111000$
 - b. $.000011$
 - c. $.110000$
 - d. $.111111$
 - e. $.101000$

Mixed Numbers

- **Covert the integer and the fraction separately**
- **Example:**
 - $5.75 = 101.11$

Examples

- **Convert the following to base 10 :**

11.01100_2

a. 3.7500

b. 3.0300

c. 3.1875

d. 3.0300

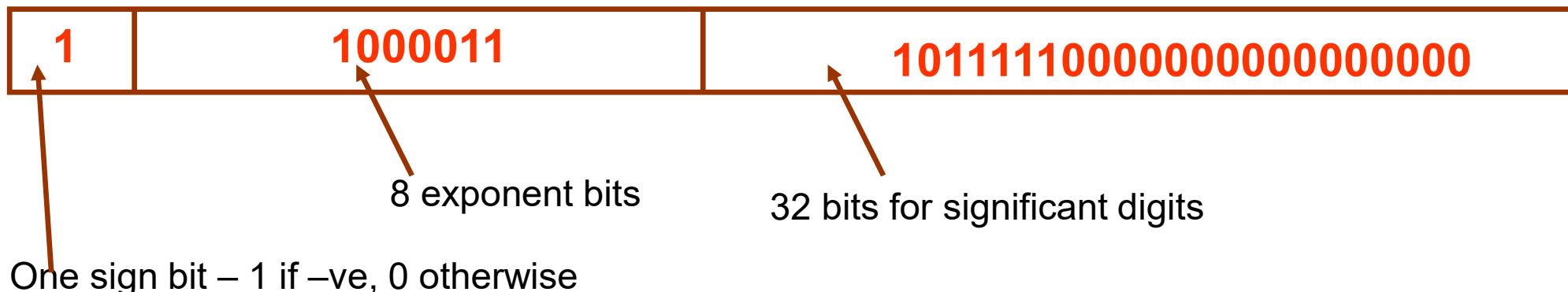
e. 3.3750

Mixed Numbers

- **Computer storage**

- The standard notation (IEEE Standard 754) for 32 bit numbers is:

- A sign bit: 1 for negative and 0 for positive
- An 8-bit exponent
 - Stored as the binary version of $127 + \text{exponent}$
 - Can store $-126:127$ as $1:254$
- 23 bits for the significant digits
- The first significant digit is always a binary 1 so this is not stored
- Example: -27.875
- $27.875 = 11011.111 = 1.1011111 \times 2^4$



Computer Storage

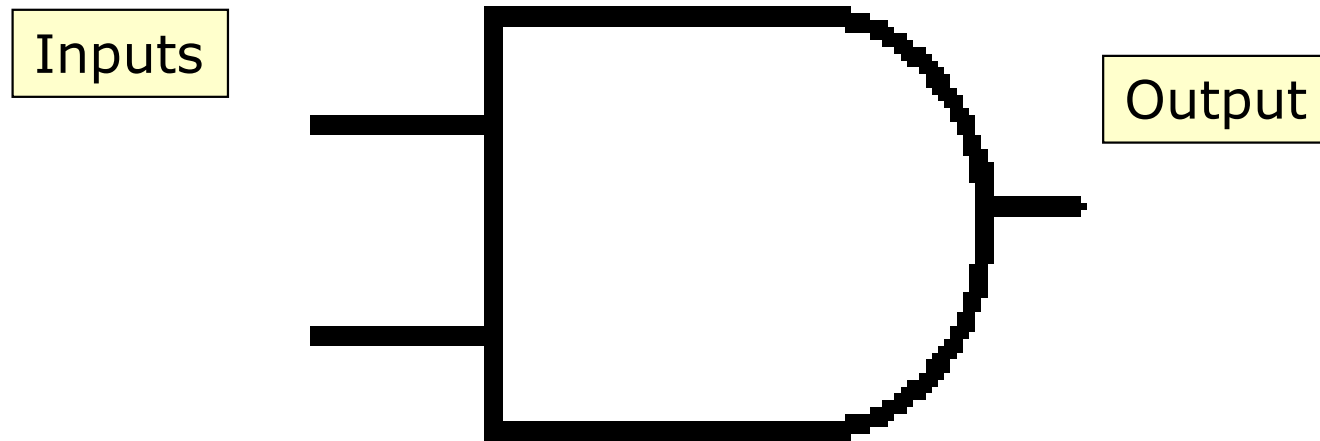
- How would the number 2.1 be stored in IEEE Standard 754 for 32 bit numbers
 - a. 1 10000001 01100110011001100110000
 - b. 0 10000000 00001100110011001100110
 - c. 0 00000001 10000110011001100110011
 - d. 1 10000000 10000000000000000000000000000
 - e. Can't be stored

- **Basic Digital logic is based on 3 primary functions (the basic gates):**
 - AND
 - OR
 - NOT

The AND function

- **The AND function:**
 - If all the inputs are high is the output is high
 - If any input is low, the output is low
- **“If this input AND this input are high, the output is high”**

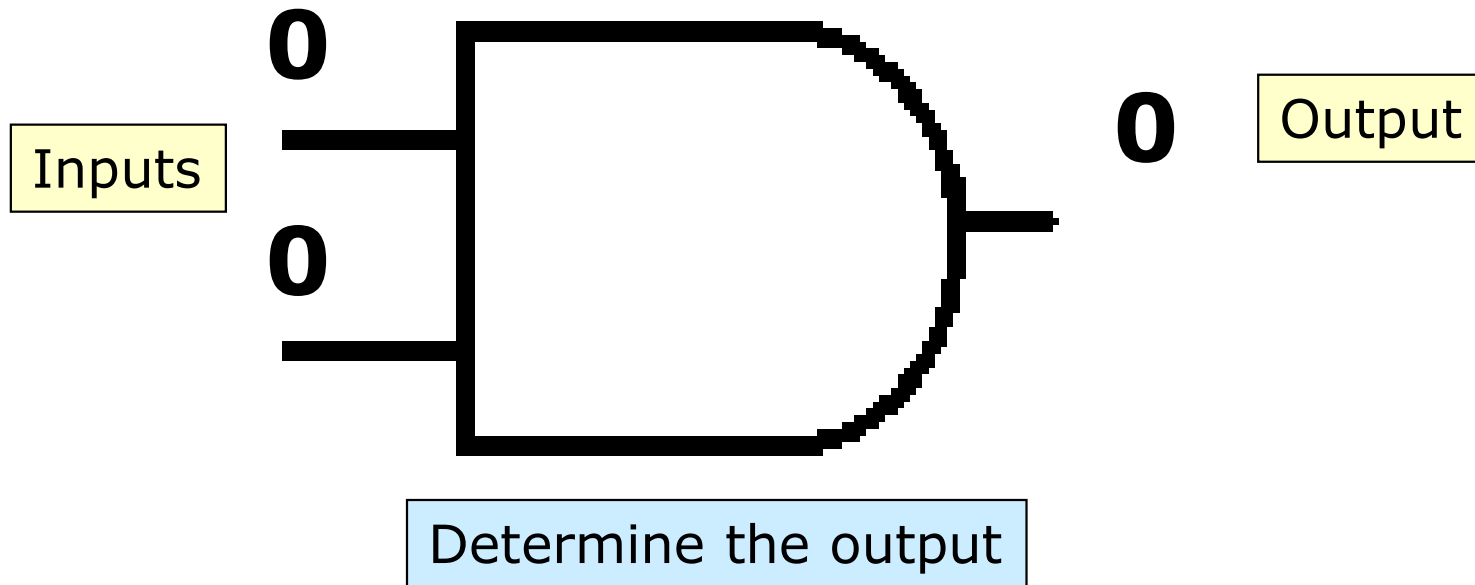
AND Logic Symbol



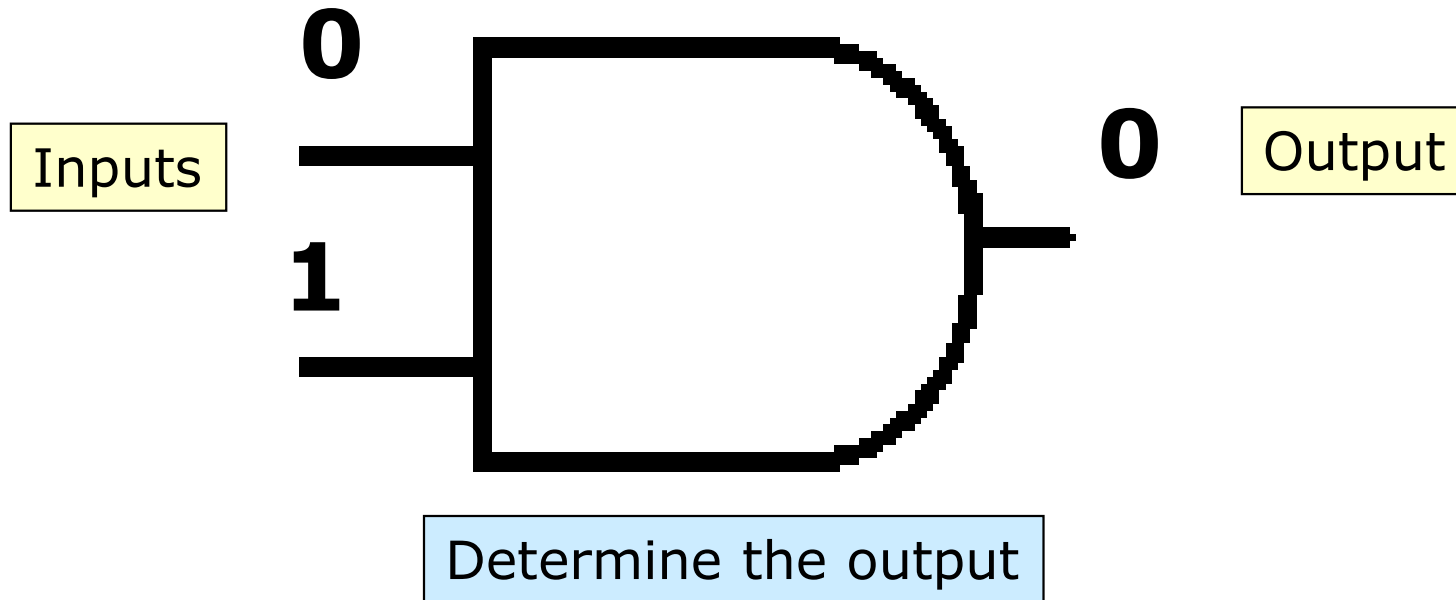
If both inputs are 1, the output is 1

If any input is 0, the output is 0

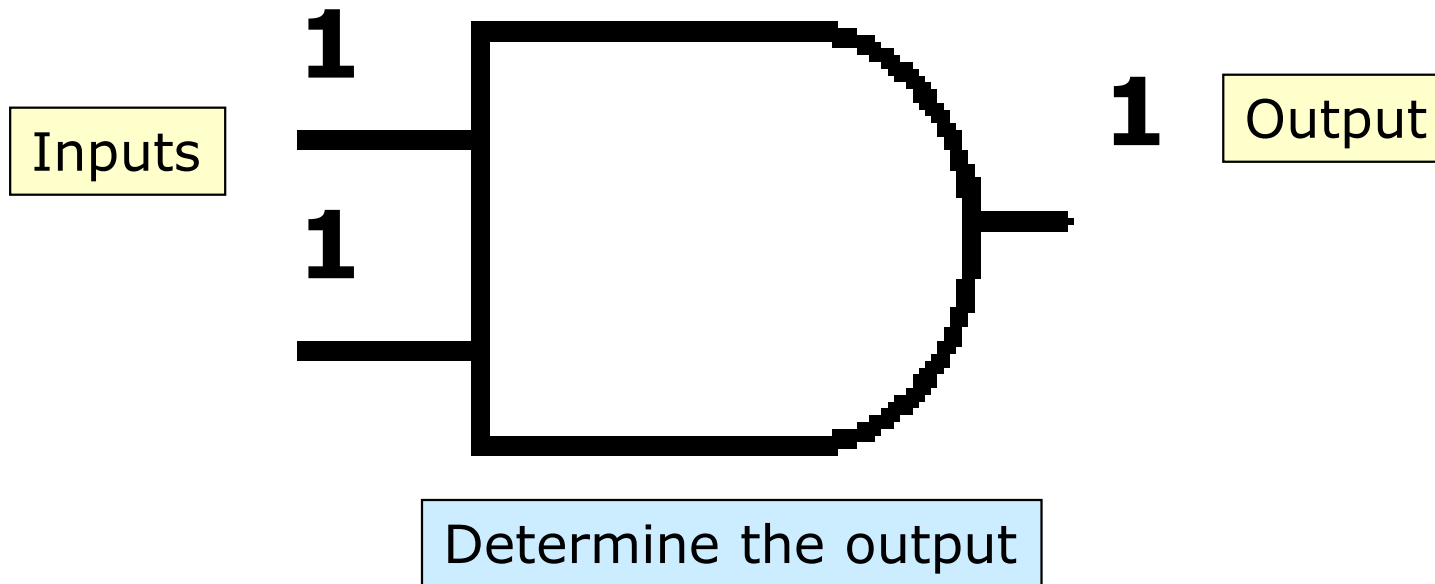
AND Logic Symbol



AND Logic Symbol



AND Logic Symbol



AND Truth Table

- To help understand the function of a digital device, a Truth Table is used:

Every possible
input combination

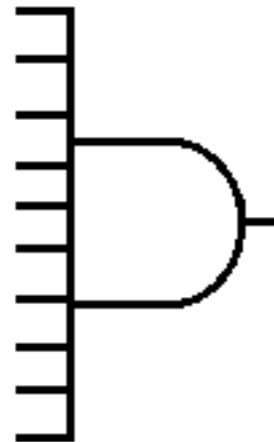
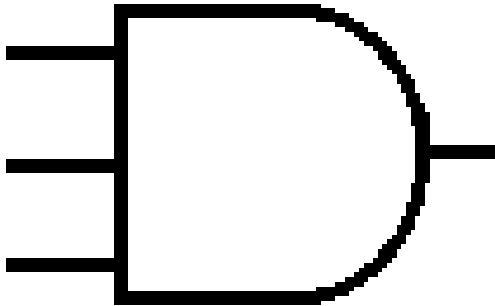


Input		Output
0	0	0
0	1	0
1	0	0
1	1	1

AND Function

AND Gates

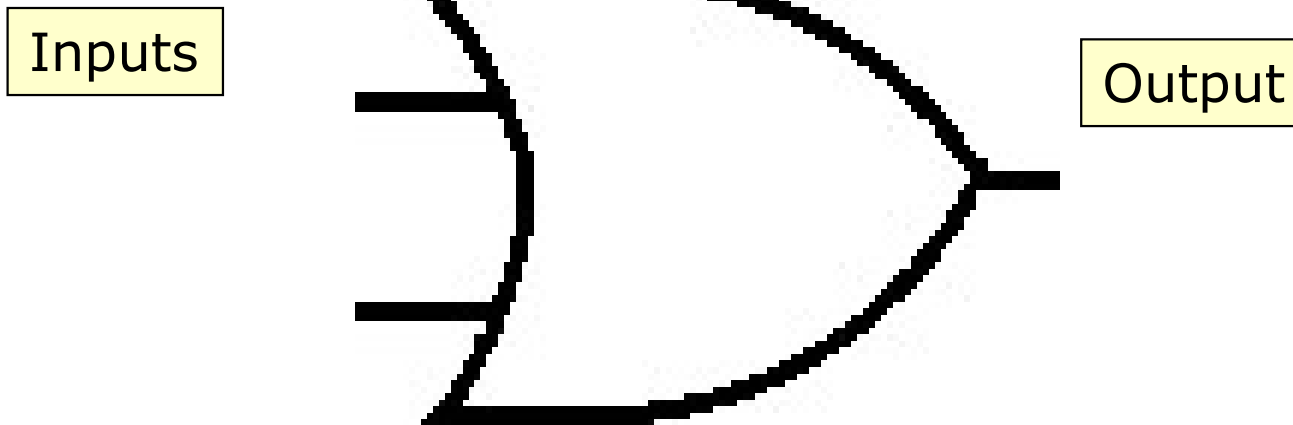
- It is possible to have AND gates with more than 2 inputs. The same logic rules apply – “if any input...”



The OR function

- **The OR function:**
 - if any input is high, the output is high
 - if all inputs are low, the output is low
- **“If this input OR this input is high, the output is high”**

OR Logic Symbol

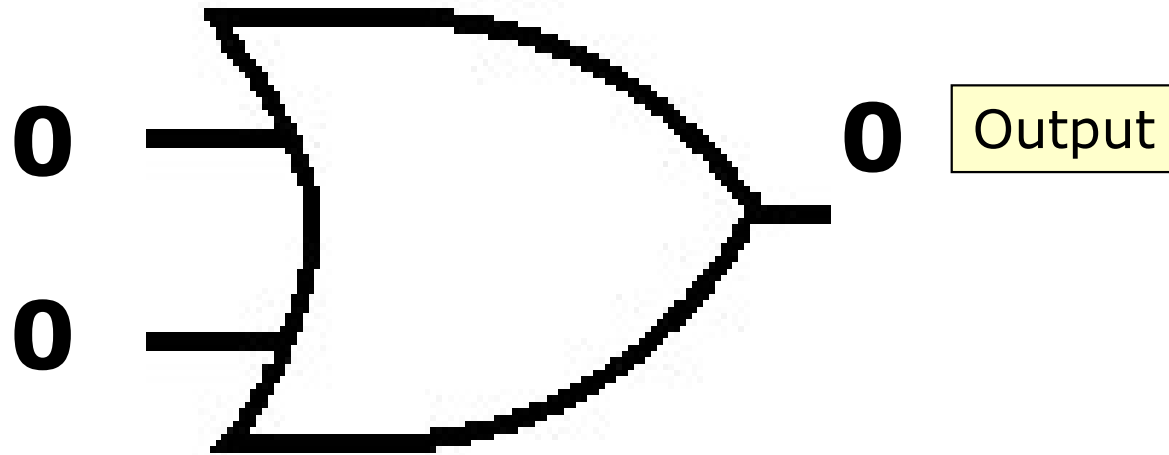


If any input is 1, the output is 1

If all inputs are 0, the output is 0

OR Logic Symbol

Inputs

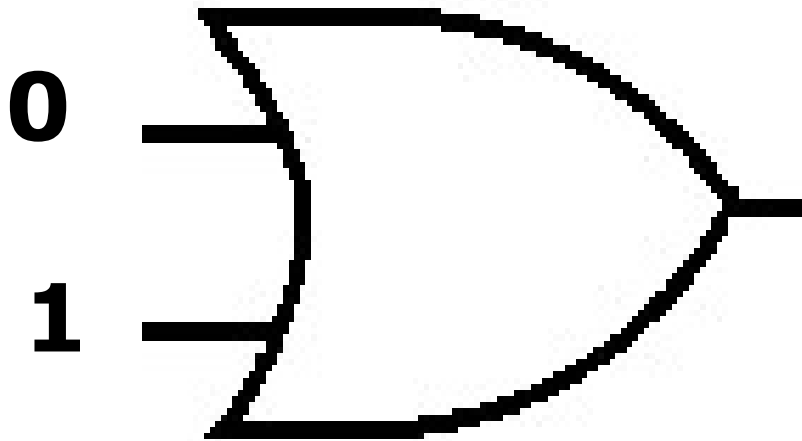


Output

Determine the output

OR Logic Symbol

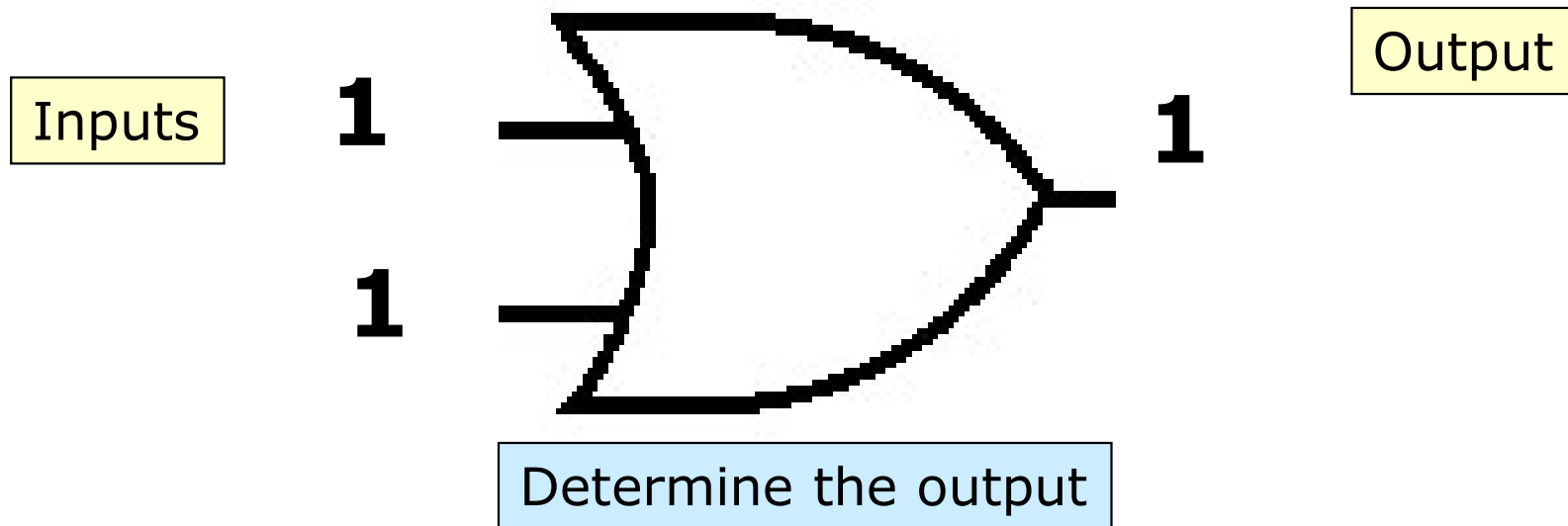
Inputs



Output

Determine the output

OR Logic Symbol



OR Truth Table

- Truth Table**

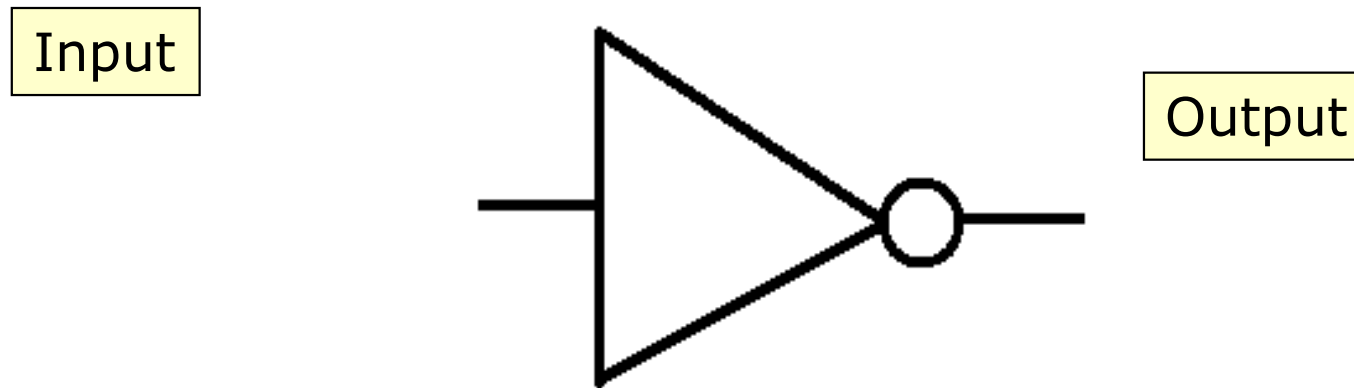
Input		Output
0	0	0
0	1	1
1	0	1
1	1	1

OR Function

The NOT function

- **The NOT function:**
 - If any input is high, the output is low
 - If any input is low, the output is high
- **“The output is the opposite state of the input”**
- **The NOT function is often called INVERTER**

NOT Logic Symbol

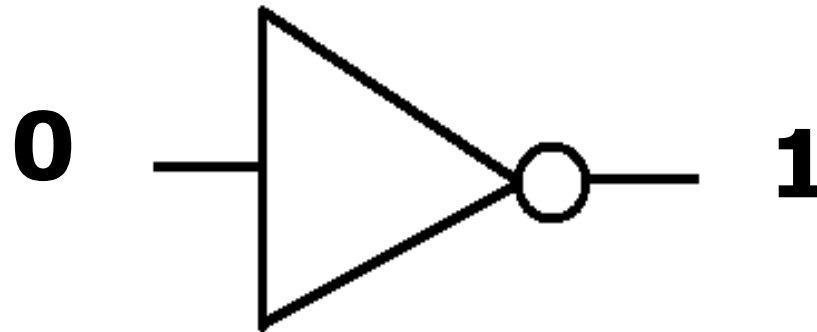


If the input is 1, the output is 0

If the input is 0, the output is 1

NOT Logic Symbol

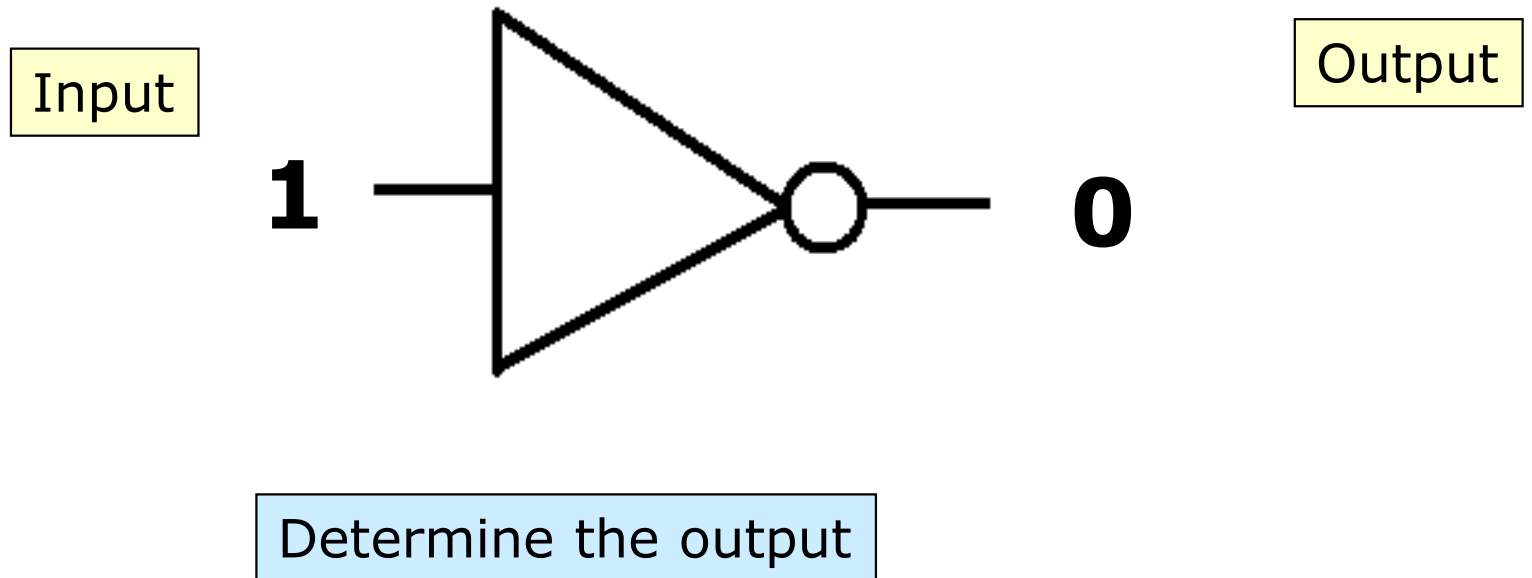
Input



Output

Determine the output

NOT Logic Symbol



Summary

OR (written as $+$)

$a + b$ (read a **OR** b) is 1 if and only if $a = 1$ **or** $b = 1$ or both

AND (written as $\bullet$ or simply two variables catenated)

$a \bullet b = ab$ (read a **AND** b) is 1 if and only if $a = 1$ **and** $b = 1$.

NOT (written as $'$)

a' (read **NOT** a) is 1 if and only if $a = 0$

Homework

- $(2536)_{10} = (?)_{16}$
- $(2536)_9 = (?)_{10}$
- $(2343)_5 = (?)_8$
- $(2AAC)_{16} = (?)_8$
- **Thank You**